Theory Interview Question

Q76. What is the DOM?

The DOM (Document Object Model) is a programming interface for web documents like HTML and XML. It represents the structure of a webpage as a tree of objects (nodes) that programming languages like JavaScript can interact with.

Simple Explanation

When a browser loads an HTML page, it converts it into a DOM tree, where:

- Each HTML tag becomes a node
- The nodes are connected in a parent-child hierarchy

Example HTML:

```
<html>
 <body>
  <h1>Hello</h1>
 </body>
</html>
```

DOM structure:

```
Document
└── html
    └── body
        └── h1
```

Key Points

1. DOM allows dynamic changes

- JavaScript can add, delete, or modify elements in real time.

```
document.querySelector("h1").textContent = "Hi!";
```

2. It is language-independent but mostly used with JavaScript

- Any language can use DOM APIs, but JS is the most common.

3. DOM represents the page as objects

- Each element has properties and methods:
    - innerHTML
    - textContent
    - style
    - appendChild()

4. Event handling

- DOM lets you handle user interactions:

```
button.addEventListener("click", () => {
 alert("Clicked!");
});
```

5. DOM is tree-based

- Helps traverse elements:
    - parentNode
    - childNodes
    - nextElementSibling

Important Concepts You Might Be Asked

- DOM vs HTML
    - HTML = static markup
    - DOM = live, interactive representation
- DOM Manipulation
    - Creating elements (createElement)
    - Updating (innerText, setAttribute)
    - Removing (remove())
- Reflow & Repaint (Performance)
    - Changing DOM can trigger layout recalculations

Q76.How do you select elements in DOM?

We can select elements in the DOM using JavaScript DOM APIs. The most commonly used methods are:

---

1. getElementById()

- Selects a single element by its id

```
const element = document.getElementById("header");
```

---

2. getElementsByClassName()

- Selects elements by class name
- Returns a live HTMLCollection

```
const items = document.getElementsByClassName("item");
```

---

3. getElementsByTagName()

- Selects elements by tag name

```
const divs = document.getElementsByTagName("div");
```

---

4. querySelector() ⭑ (Most commonly used)

- Returns the first matching element
- Uses CSS selectors

```
const element = document.querySelector(".item");
const element2 = document.querySelector("#header");
```

---

5. querySelectorAll()

- Returns all matching elements
- Returns a NodeList

```
const elements = document.querySelectorAll(".item");
```

Important Differences (Interview Highlight)

- getElementById → single element
- querySelector → first match
- querySelectorAll → all matches (NodeList)
- getElementsByClassName → live collection

Q77.Difference between querySelector and getElementById?

---

1. Basic Difference

- getElementById()

    - Selects element using id only
- querySelector()
    - Selects element using any CSS selector

---

2. Syntax

```
// getElementById
const el1 = document.getElementById("header");

// querySelector
const el2 = document.querySelector("#header");
```

---

3. Flexibility

- getElementById → ❌ Only works with id
- querySelector → ✅ Works with:
    - id (#id)
    - class (.class)
    - tag (div)
    - attributes (input[type="text"])

---

4. Return Value

- Both return only the first matching element
- But:
    - getElementById → always one unique element (IDs should be unique)
    - querySelector → first match if multiple exist

---

5. Performance

- getElementById → ⚡ Slightly faster
- querySelector → 🐢 Slightly slower (because it parses CSS selectors)

👉 In real-world apps, the difference is usually negligible.

---

6. Use Case

- Use getElementById when:
    - You know the exact ID and want maximum performance
- Use querySelector when:

- You need flexibility and cleaner, modern code

Q78.What is event delegation?

What is Event Delegation?

Event Delegation is a technique in the DOM where you attach a single event listener to a parent element instead of adding event listeners to multiple child elements.

It works because of event bubbling (events propagate from child → parent).

Simple Example

Instead of doing this ❌:

```
document.querySelectorAll("li").forEach(li => {
 li.addEventListener("click", () => {
   console.log("Item clicked");
 });
});
```

We do this

```
document.querySelector("ul").addEventListener("click", (event) => {
 if (event.target.tagName === "LI") {
   console.log("Item clicked");
 }
});
```

How it Works

- Click happens on <li>
- Event bubbles up to <ul>
- Parent handler catches it using event.target

Advantages (Important for Interview)

1. Better Performance

- Fewer event listeners → less memory usage

2. Handles Dynamic Elements

- Works for elements added later (no need to reattach listeners)

3. Cleaner Code

- Centralized event handling

Key Concepts to Mention

- event.target → actual clicked element
- event.currentTarget → element with listener
- Event bubbling

Q79. How do you add/remove event listeners?

How do you add/remove event listeners?

1. Adding Event Listeners

We use addEventListener():

```
const button = document.getElementById("btn");

function handleClick() {
 console.log("Button clicked");
}

button.addEventListener("click", handleClick);
```

---

2. Removing Event Listeners

We use removeEventListener():

```
button.removeEventListener("click", handleClick);
```

Important Rules (Very Important for Interview)

1. Same Function Reference Required ⚠

- You must pass the same function reference to remove it

❌ This will NOT work:

```
button.removeEventListener("click", () => {
 console.log("clicked");
});
```

✅ Correct way:

```
function handleClick() {
 console.log("clicked");
}

button.addEventListener("click", handleClick);
button.removeEventListener("click", handleClick);
```

---

2. Same Event Type & Options

- Event type (click) must match
- Capture option must also match if used

Q80.What is preventDefault()?

What is preventDefault()?

Theory Explanation

preventDefault() is a method of the Event interface in JavaScript. It is used to cancel the default action that the browser performs when a specific event occurs.

Every HTML element has some predefined behavior (called default behavior). When an event is triggered on that element, the browser automatically performs that action unless it is explicitly prevented.

preventDefault() allows developers to override or control that behavior.

Key Concept

- It works on the event object
- It only prevents the default browser action
- It does NOT stop event propagation (bubbling/capturing)

Syntax

```
event.preventDefault();
```

Practical Examples

1. Prevent Form Submission (Most Important Use Case)

```
<form id="form">
 <input type="text" id="name" />
 <button type="submit">Submit</button>
</form>
document.getElementById("form").addEventListener("submit", function(e) {
 e.preventDefault();
 console.log("Form submission prevented");
});
```

✅ Explanation:

- By default, form submits and reloads the page
- Using preventDefault(), we stop reload
- Useful for:
  - Form validation
  - API calls (AJAX)

---

2. Prevent Link Navigation

```
<a href="https://google.com" id="link">Go to Google</a>
document.getElementById("link").addEventListener("click", function(e) {
 e.preventDefault();
 console.log("Navigation stopped");
});
```

✅ Explanation:

- Normally redirects to another page
- preventDefault() stops navigation

---

3. Custom Validation Example

```
form.addEventListener("submit", function(e) {
 const value = document.querySelector("input").value;

 if (value === "") {
   e.preventDefault();
   alert("Field cannot be empty");
```

```
 }
});
```

---

4. Prevent Right-Click Menu

```
document.addEventListener("contextmenu", function(e) {
 e.preventDefault();
});
```

Important Points (Interview Highlights)

- Works only on cancelable events
- Does not stop bubbling → use stopPropagation() for that
- Commonly used in:
  - Forms
  - Links
  - User interactions

Q81. What is localStorage and sessionStorage?

---

localStorage vs sessionStorage – In Depth

Both localStorage and sessionStorage are part of the Web Storage API, which provides a client-side storage mechanism in modern browsers. They allow you to store key-value pairs that persist in the browser without involving a server.

---

1. Key Features of Web Storage

- Stores data as strings (use JSON.stringify/JSON.parse for objects).
- Larger storage size than cookies (typically 5–10 MB).
- Accessible via JavaScript.
- Data is scoped by origin (protocol + domain + port).

---

2. localStorage

Definition:
localStorage stores data permanently in the browser until it is explicitly deleted. Closing the browser does not clear it.

Properties:

- Persistent across browser sessions
- Shared across all tabs/windows of the same origin
- Maximum storage ~5–10 MB
- Only string values allowed

Usage Example

```
<script>
 // Save data
 localStorage.setItem("username", "John");

 // Retrieve data
 const user = localStorage.getItem("username");
 console.log(user); // John

 // Remove a specific item
 localStorage.removeItem("username");

 // Clear all localStorage
 localStorage.clear();

 // Store objects
 const userObj = { name: "John", age: 25 };
 localStorage.setItem("user", JSON.stringify(userObj));

 const retrievedUser = JSON.parse(localStorage.getItem("user"));
 console.log(retrievedUser.name); // John
</script>
```

Use Cases:

- Persist user preferences (theme, language)
- Keep login info (non-sensitive data)
- Save app settings

---

3. sessionStorage

Definition:
 sessionStorage stores data temporarily. It lasts only for the duration of the page session (until the tab is closed).

Properties:

- Data cleared when tab/window closes
- Not shared across different tabs (even same origin)

- Maximum storage ~5–10 MB
- Only string values allowed

Usage Example

```
<script>
 // Save session data
 sessionStorage.setItem("sessionId", "12345");

 // Retrieve session data
 const sessionId = sessionStorage.getItem("sessionId");
 console.log(sessionId); // 12345

 // Remove data
 sessionStorage.removeItem("sessionId");

 // Clear all sessionStorage
 sessionStorage.clear();
</script>
```

Use Cases:

- Store temporary session data
- Multi-step forms
- Temporary user states

---

4. Key Differences

Feature	localStorage	sessionStorage
Storage duration	Permanent (until manually cleared)	Until tab/window closes
Scope	Shared across all tabs of same origin	Tab-specific (not shared)
Storage limit	~5–10 MB	~5–10 MB
Accessibility	All pages of same origin	Only in the current tab/page
Example	User theme preference	Multi-step form data

---

5. Important Notes (Interview Points)

- Both store data as strings only → use JSON.stringify/JSON.parse for objects.
- Same-origin policy applies → only accessible from same domain/port/protocol.
- Not secure for sensitive info → use for preferences, not passwords.
- Faster than cookies → no server requests.

Q81.What is localStorage vs sessionStorage?

Key Differences

Feature	localStorage	sessionStorage
Storage duration	Permanent (until manually cleared)	Until tab/window closes
Scope	Shared across all tabs of same origin	Tab-specific (not shared)
Storage limit	~5–10 MB	~5–10 MB
Accessibility	All pages of same origin	Only in the current tab/page
Example	User theme preference	Multi-step form data

Q82. What is the difference between cookies and localStorage?

Difference Between Cookies and localStorage

Both **cookies** and **localStorage** are ways to store data in the browser, but they differ significantly in purpose, size, and behavior.

---

1. Storage Size

Feature	**Cookies**	**localStorage**
Size	~4 KB	~5–10 MB

localStorage can store much more data than cookies.

---

2. Lifespan

- **Cookies** → Can be session-based or have an expiration date (expires or max-age).
- **localStorage** → Persistent until explicitly cleared by user or JavaScript.

---

3. Server Communication

- **Cookies** → Sent automatically with **every HTTP request** to the server (used for authentication, sessions).
- **localStorage** → Only accessible via **JavaScript on client-side**; **never sent to server automatically**.

---

4. Scope

- **Cookies** → Can be scoped to **domain/path**, sent to server requests within that scope.
- **localStorage** → Scoped to **origin (protocol + domain + port)**, shared across pages of same origin, not sent to server.

---

5. Ease of Access / Performance

Feature	Cookies	localStorage
Access	Document.cookie (manual parsing)	Simple API: setItem, getItem, removeItem
Performance	Slower (sent to server every request)	Faster (client-only, no server request)

---

6. Security

- **Cookies** → Can be **HttpOnly**, making them inaccessible to JavaScript (more secure for auth tokens).
- **localStorage** → Fully accessible to JavaScript, vulnerable to **XSS attacks**.

---

7. Example

Cookies

```
// Set cookie
document.cookie = "username=John; expires=Fri, 31 Dec 2026 23:59:59 GMT; path=/";

// Get cookie
console.log(document.cookie);
```

localStorage

```
// Set localStorage
localStorage.setItem("username", "John");

// Get localStorage
console.log(localStorage.getItem("username"));
```

---

8. Quick Comparison Table

Feature	Cookies	localStorage
Storage Size	~4 KB	5–10 MB
Lifespan	Session or expiration date	Persistent until cleared
Sent to Server	Yes	No
Access	JS + document.cookie	JS API (getItem, setItem)
Security	Can be HttpOnly	Accessible via JS (vulnerable to XSS)
Scope	Domain/path	Origin (protocol + domain + port)

JSON Questions

Q82. What is JSON and why is it used?

1. Definition (Theory)

JSON stands for JavaScript Object Notation. It is a lightweight, text-based format for exchanging data between a client and a server.

- It is language-independent, meaning it can be used with JavaScript, Python, Java, PHP, etc.
- It is human-readable and easy to parse by machines.
- JSON represents data as key-value pairs.

---

2. Why JSON is Used

1. Data Interchange – To send structured data between client and server over HTTP.
2. Lightweight – Less verbose than XML, reducing network overhead.
3. Readable and Easy to Parse – Can be easily read by humans and parsed by programming languages.
4. Supports Complex Structures – Can handle objects, arrays, strings, numbers, booleans, and null.
5. Compatible with REST APIs – Most modern web services use JSON as the standard format.

---

3. JSON Syntax Rules

- Data is written as key-value pairs
- Keys must be in double quotes
- Strings must be in double quotes
- Values can be:
    - String
    - Number
    - Boolean (true/false)
    - Null
    - Array
    - Object

Example:

```
{

 "name": "John",

 "age": 25,

 "isStudent": true,

 "skills": ["JavaScript", "React"]

}
```

---

4. Practical Use Example

```
const jsonData = '{"name":"John","age":25}';


// Convert JSON string to JavaScript object

const obj = JSON.parse(jsonData);

console.log(obj.name); // John


// Convert JS object to JSON string

const newJson = JSON.stringify({ name: "Alice", age: 30 });

console.log(newJson); // '{"name":"Alice","age":30}'
```

Q83. What is the difference between JSON and a JavaScript object?

1. Definition
    - JSON: A lightweight, text-based format used to exchange data between client and server.
    - JavaScript Object: An in-memory data structure used to store and manipulate data in JavaScript code.
2. Data Type / Format
    - JSON: Always a string.
    - JS Object: A live object in memory.
3. Key Syntax
    - JSON: Keys must be in double quotes.
    - JS Object: Keys may or may not have quotes.
4. Allowed Values
    - JSON: Only strings, numbers, booleans, null, arrays, objects. Cannot include functions, undefined, or symbols.
    - JS Object: Can store any valid JS value, including functions, undefined, symbols, arrays, and objects.
5. Usage
    - JSON: Used mainly for data exchange, e.g., API responses, localStorage, or sending data to a server.
    - JS Object: Used for program logic and manipulation in JavaScript code.
6. Parsing / Conversion
    - JSON: Needs JSON.parse() to convert to a JS object and JSON.stringify() to convert back to string.
    - JS Object: Can be used directly without conversion.
7. Mutability / Behavior

- JSON: Immutable as a string until parsed into an object.
- JS Object: Mutable and can be manipulated directly in code.

Example

```
const jsObj = { name: "John", greet: () => console.log("Hi") }; // JS Object

const jsonStr = '{"name":"John"}'; // JSON string


// Convert JSON to JS object

const parsedObj = JSON.parse(jsonStr);

console.log(parsedObj.name); // John
```

Q84.How do you convert a JSON string to a JavaScript object?

1. Using JSON.parse() Method

The JSON.parse() method is used to convert a JSON string into a JavaScript object.

Syntax

```
JSON.parse(jsonString)
```

- jsonString – The JSON-formatted string to convert.
- Returns a JavaScript object.

2. Example

```
const jsonString = '{"name":"John","age":25,"isStudent":true}';

// Convert JSON string to JavaScript object

const jsObject = JSON.parse(jsonString);

console.log(jsObject.name);  // John

console.log(jsObject.age);   // 25

console.log(jsObject.isStudent); // true
```

3. Using with Arrays

```
const jsonArray = '[{"id":1,"name":"Alice"},{"id":2,"name":"Bob"}]';

const jsArray = JSON.parse(jsonArray);

console.log(jsArray[0].name); // Alice
```

4. Important Points (Interview Highlights)

1. The JSON string must be properly formatted:
    - Keys must be in double quotes "key".
    - Strings must use double quotes "value".
2. Errors: If the JSON string is invalid, JSON.parse() will throw a SyntaxError.
3. Use Case: Commonly used when receiving API responses or retrieving JSON from localStorage/sessionStorage.

Q85. How do you convert a JavaScript object to a JSON string?

1. Using JSON.stringify() Method

The JSON.stringify() method is used to convert a JavaScript object or array into a JSON string.

Syntax

JSON.stringify(value, replacer, space)

- value – The JavaScript object or array to convert.
- replacer *(optional)* – A function or array to filter/select properties.
- space *(optional)* – Adds indentation/formatting for readability.

---

2. Example: Basic Object

```
const jsObject = { name: "John", age: 25, isStudent: true };



// Convert JS object to JSON string

const jsonString = JSON.stringify(jsObject);



console.log(jsonString);

// Output: '{"name":"John","age":25,"isStudent":true}'
```

---

3. Example: With Arrays

```
const jsArray = [
 { id: 1, name: "Alice" },
 { id: 2, name: "Bob" }
];

const jsonString = JSON.stringify(jsArray);
console.log(jsonString);
// Output: '[{"id":1,"name":"Alice"},{"id":2,"name":"Bob"}]'
```

---

4. Example: Using replacer and space

```
const jsObject = { name: "John", age: 25, password: "12345" };

// Exclude password and pretty print
const jsonString = JSON.stringify(jsObject, ["name","age"], 2);

console.log(jsonString);
/* Output:
{
 "name": "John",
 "age": 25
}
*/
```

---

5. Important Points (Interview Highlights)

1. Converts JavaScript objects or arrays into JSON string.
2. Functions, undefined, and symbols are ignored during conversion.
3. Useful for sending data to APIs, storing in localStorage/sessionStorage, or logging structured data.
4. JSON.stringify() ensures double quotes around keys, which is valid JSON syntax.

Q86. Can JSON store functions or undefined values?

---

1. Theory / Explanation

- JSON (JavaScript Object Notation) is a text-based data format designed for data exchange, not for storing executable code.
- JSON can only store data types that can be represented as text:
  - Strings
  - Numbers
  - Booleans (true / false)
  - Null
  - Objects
  - Arrays
- Functions, undefined, and symbols cannot be stored in JSON. If you try, they will be ignored or cause errors when parsing/stringifying.

---

2. Example

```
const jsObj = {
 name: "John",
 greet: function() { console.log("Hi"); },
 age: undefined
};

const jsonString = JSON.stringify(jsObj);
console.log(jsonString);
// Output: '{"name":"John"}'
// greet (function) and age (undefined) are ignored
```

---

3. Key Points for Interviews

1. JSON is data-only, not code.
2. Functions or undefined cannot be serialized in JSON.
3. Attempting to include them in JSON.stringify() will ignore these properties silently.
4. To store a function-like behavi**or, you need to convert it to a string and handle it separately.**

Web Services Questions

Q87. What is a web service?

1. Definition of Web Service

A Web Service is a software system that allows different applications to communicate with each other over a network (usually the Internet) using standardized protocols like HTTP, XML, or JSON.

- It enables machine-to-machine communication.
- Platform-independent: Works across different operating systems, programming languages, and devices.

---

2. Why Use Web Services

1. Interoperability
    - Allows applications written in different languages to communicate.
    - Example: A Java application can communicate with a Python application.
2. Reusability
    - Services can be used by multiple applications without rewriting code.
3. Platform and Language Independent
    - Works across Windows, Linux, macOS, and multiple programming languages.
4. Integration
    - Integrates applications over the web (e.g., connecting a payment gateway with an e-commerce website).
5. Standardized Communication
    - Uses standards like HTTP, XML, JSON, SOAP, or REST for data exchange.

---

3. Types of Web Services

1. SOAP (Simple Object Access Protocol)
    - Protocol-based, uses XML for messaging.
    - More rigid, supports security and transaction standards.
2. REST (Representational State Transfer)
    - Architectural style, usually uses JSON or XML.
    - Lightweight, stateless, and widely used for modern web/mobile apps.

---

4. Example Use Case

- A weather app requests data from a weather API:

  Browser/App → HTTP Request → Web Service → Returns JSON → Display Weather

- A payment gateway communicates with your e-commerce site to confirm transactions.

Q88.What are the main types of web services?

1. SOAP Web Services (Simple Object Access Protocol)

Definition:
SOAP is a protocol-based web service that uses XML for messaging between client and server.

Key Features:

- Highly standardized and strict.
- Supports security, transactions, and ACID compliance.
- Uses WSDL (Web Services Description Language) to describe the service.
- Works over HTTP, SMTP, TCP, and other protocols.

Use Cases:

- Banking applications
- Enterprise systems requiring high security and transaction management

Example:

- Sending a SOAP request in XML to fetch account details.

---

2. REST Web Services (Representational State Transfer)

Definition:
REST is an architectural style, not a protocol, that allows communication via HTTP methods (GET, POST, PUT, DELETE) and usually uses JSON or XML for data transfer.

Key Features:

- Lightweight, faster, and easier to implement.
- Stateless: Each request contains all the information needed.
- Uses URIs to access resources.
- Platform-independent and language-agnostic.

Use Cases:

- Modern web/mobile applications
- Social media APIs (Facebook, Twitter)
- Payment gateway APIs

Q89. What is the difference between SOAP and REST?

SOAP and REST are both used for web services, but they differ in protocol, format, and usage.

---

1. Definition

- SOAP (Simple Object Access Protocol): A protocol-based web service that uses XML for messaging and has strict standards.
- REST (Representational State Transfer): An architectural style that uses HTTP methods and usually JSON for communication.

---

2. Key Differences (Point-Wise)

Feature	SOAP	REST
Type	Protocol	Architectural style
Message Format	XML only	JSON, XML, HTML, plain text
Transport	Can use HTTP, SMTP, TCP, JMS	Only HTTP/HTTPS
State	Can be Stateful or Stateless	Stateless only
Ease of Use	Complex, verbose	Simple, lightweight
Performance	Slower due to XML and overhead	Faster due to lightweight JSON/HTTP

Security	WS-Security, encryption, built-in	Relies on HTTPS, OAuth, JWT
Standards	Strict standards (WSDL, SOAP envelopes)	Less strict, flexible URI-based endpoints
Caching	Difficult	Supports HTTP caching
Error Handling	Built-in standardized fault structure	Uses HTTP status codes (404, 500, etc.)

REST API Questions

Q90. What is REST API?

1. Definition

REST API (Representational State Transfer Application Programming Interface) is an architectural style for building web services that allows applications to communicate over HTTP using standard methods like GET, POST, PUT, and DELETE.

- It is stateless, meaning each request contains all information needed.
- REST APIs typically exchange data in JSON or XML format.
- Focuses on resources (identified by URIs) rather than operations.

---

2. Key Features of REST API

1. Stateless: Each request from client to server contains all required information; the server does not store client session state.
2. Client-Server Architecture: Separates client (frontend) and server (backend) responsibilities.
3. Cacheable: Responses can be cached to improve performance.
4. Uniform Interface: Resources are accessed using standard HTTP methods.
5. Resource-Based: Every resource has a unique URI.
6. Lightweight: Often uses JSON, making it faster and easier than SOAP.

---

3. HTTP Methods in REST API

Method	Purpose
GET	Retrieve data from the server
POST	Create a new resource
PUT	Update an existing resource
PATCH	Partially update a resource
DELETE	Remove a resource

---

4. Example

GET Request to fetch a user by ID:

```
GET https://api.example.com/users/123
```

Response (JSON):

```
{
 "id": 123,
 "name": "John Doe",
 "email": "john@example.com"
}
```

Q91. What are the common HTTP methods used in REST?

REST APIs use standard HTTP methods to perform operations on resources. The most common methods are:

---

1. GET

- Purpose: Retrieve data from the server.
- Idempotent: Yes (repeating the request has the same effect).
- Example:

  GET https://api.example.com/users/123

- Returns user details in JSON or XML.

---

2. POST

- Purpose: Create a new resource on the server.
- Idempotent: No (multiple calls may create multiple resources).
- Example:

  POST https://api.example.com/users

  Content-Type: application/json

  {

  "name": "Alice",

  "email": "alice@example.com"

  }

---

3. PUT

- Purpose: Update an existing resource completely.
- Idempotent: Yes (multiple calls produce the same result).
- Example:

  PUT https://api.example.com/users/123

  Content-Type: application/json

  {

  "name": "Alice Smith",

```
 "email": "alice@example.com"

}
```

---

4. PATCH

- Purpose: Partially update a resource.
- Idempotent: Yes (same effect if repeated).
- Example:

```
PATCH https://api.example.com/users/123

Content-Type: application/json



{

 "email": "alice.smith@example.com"

}
```

---

5. DELETE

- Purpose: Delete a resource from the server.
- Idempotent: Yes (deleting an already deleted resource has no further effect).
- Example:

```
DELETE https://api.example.com/users/123
```

---

6. Optional Methods

- HEAD: Retrieve headers only, no body.
- OPTIONS: Check which HTTP methods are allowed on a resource.

Q92. What is statelessness in REST API?

1. Definition

Statelessness in REST API means that each client request to the server must contain all the information necessary to understand and process the request.

- The server does not store any client session information between requests.
- Every request is independent.

---

2. Key Points

1. No server-side session storage:
    - The server does not remember previous requests.
2. Client responsibility:
    - The client must include authentication, parameters, and context with every request.
3. Scalability:
    - Statelessness allows servers to handle more requests efficiently because there's no session tracking.
4. Simplifies architecture:
    - No need for session management on the server.

---

3. Example

Stateless Request:

GET https://api.example.com/users/123

Authorization: Bearer <token>

- The server can respond correctly without knowing any previous requests.

Stateful Example (Not RESTful):

- If the server required storing login session info and relied on previous requests, it would not be stateless.

Q93. What is the difference between URI and URL in REST?

In REST APIs, URI and URL are related concepts but have distinct purposes.

---

1. Definition

- URI (Uniform Resource Identifier):

    - A string that identifies a resource on the web, either by location, name, or both.
    - Example: /users/123
- URL (Uniform Resource Locator):
    - A type of URI that specifically identifies the location of a resource on the web.
    - Example: https://api.example.com/users/123

---

2. Key Differences (Point-Wise)

Feature	URI	URL
Definition	Identifies a resource	Identifies a resource and its location
Type	Generic identifier	Subset of URI
Information	May or may not include location	Always includes location (protocol + domain + path)
Example	/users/123	https://api.example.com/users/123
Usage in REST	Used to define resource endpoints	Used to access the resource via HTTP

---

3. Example in REST API

- URI (Resource Identifier):

  /users/123

- URL (Access the resource over the web):

  https://api.example.com/users/123

Q94. What is the difference between REST and SOAP web services?

REST and SOAP are two popular approaches to web services, but they differ in architecture, format, and use cases.

---

1. Key Differences (Point-Wise)

Feature	SOAP	REST
Full Form	Simple Object Access Protocol	Representational State Transfer
Type	Protocol	Architectural style
Data Format	XML only	JSON, XML, HTML, or plain text (JSON commonly used)
Transport Protocol	Can use HTTP, SMTP, TCP, JMS	Only HTTP/HTTPS
State	Can be Stateful or Stateless	Stateless only
Ease of Use	Complex, verbose, requires strict standards	Simple, lightweight, flexible
Security	Built-in WS-Security, supports encryption and authentication	Relies on HTTPS, OAuth, JWT
Performance	Slower due to XML and overhead	Faster due to lightweight data format
Caching	Difficult	Easy, supports HTTP caching

Standards	Strict standards with WSDL	Less strict, URI-based endpoints
Use Cases	Enterprise apps, banking, high-security transactions	Web/mobile apps, social media APIs, public APIs

---

2. Example Use Case

- SOAP: A banking application sending transaction details securely.
- REST: A weather app fetching weather data from a public API in JSON format.

MCQ Question on JavaScript

---

1. What does DOM stand for?

A. Data Object Model
B. Document Object Model
C. Digital Object Management
D. Document Oriented Method
Answer: B
Explanation: DOM represents HTML as a tree structure for JavaScript interaction.

---

2. What does document.querySelector("h1") return?

A. All h1 elements
B. First h1 element
C. Last h1 element
D. None
Answer: B
Explanation: Returns the first matching element.

---

3. Which method returns a live collection?

A. querySelectorAll
B. getElementById
C. getElementsByClassName

D. querySelector
Answer: C
Explanation: HTMLCollection is live and updates automatically.

---

4. What does this do?

```
document.querySelector("h1").textContent = "Hi";
```

A. Deletes h1
B. Changes text
C. Adds new element
D. Hides element
Answer: B
Explanation: Updates text inside <h1>.

---

5. Which is fastest?

A. querySelector
B. getElementById
C. querySelectorAll
D. getElementsByTagName
Answer: B
Explanation: Direct lookup by ID is fastest.

---

6. What is event delegation?

A. Multiple listeners
B. Parent handles child events
C. Removing events
D. Creating events
Answer: B
Explanation: Uses event bubbling.

---

7. What is event.target?

A. Parent element
B. Current listener
C. Clicked element
D. Root element
Answer: C
Explanation: The actual element clicked.

---

8. Which method adds an event?

A. addEvent
B. attachEvent
C. addEventListener
D. setEvent
Answer: C

---

9. Why might removeEventListener fail?

A. Wrong syntax
B. Different function reference
C. Wrong event
D. All
Answer: D

---

10. What does preventDefault() do?

A. Stops bubbling
B. Stops default behavior
C. Removes event
D. Stops JS
Answer: B

---

11. Does preventDefault() stop bubbling?

A. Yes
B. No
Answer: B
Explanation: Use stopPropagation() instead.

---

12. Where is localStorage stored?

A. Server
B. Browser
C. Database
D. Cache only
Answer: B

---

13. localStorage data lasts until?

A. Tab close
B. Browser close
C. Manual delete
D. 1 day
Answer: C

---

14. sessionStorage lasts until?

A. Forever
B. Tab closes
C. Browser restart
D. Server reset
Answer: B

---

15. Storage format in Web Storage?

A. JSON
B. Object
C. String
D. Array
Answer: C

---

16. Which is NOT safe for sensitive data?

A. Cookies
B. localStorage
C. sessionStorage
D. All
Answer: D

---

17. Cookie size limit?

A. 10 MB
B. 5 MB
C. 4 KB

D. Unlimited
Answer: C

---

18. JSON stands for?

A. JavaScript Object Notation
B. Java Syntax Object Notation
C. JavaScript Output Network
D. None
Answer: A

---

19. JSON keys must be?

A. Single quotes
B. Double quotes
C. No quotes
D. Numbers
Answer: B

---

20. Which is valid JSON?

A. {name: "John"}
B. {'name': "John"}
C. {"name": "John"}
D. {name: John}
Answer: C

---

21. Convert JSON → JS object?

A. JSON.stringify()
B. JSON.parse()
C. JSON.convert()
D. JSON.toObject()
Answer: B

---

22. Convert JS object → JSON?

A. JSON.parse()
B. JSON.stringify()
C. JSON.toString()
D. JSON.convert()
Answer: B

---

23. What happens to functions in JSON.stringify()?

A. Error
B. Converted
C. Ignored
D. Stored
Answer: C

---

24. REST API uses which protocol?

A. FTP
B. HTTP
C. TCP only
D. SMTP
Answer: B

---

25. Which HTTP method retrieves data?

A. POST
B. PUT
C. GET
D. DELETE
Answer: C

---

26. Which method creates data?

A. GET
B. POST
C. DELETE
D. PATCH
Answer: B

---

27. What does PUT do?

A. Create
B. Delete
C. Update completely
D. Read
Answer: C

---

28. REST APIs are?

A. Stateful
B. Stateless
C. Secure only
D. XML-based only
Answer: B

---

29. What is a URI?

A. Location only
B. Identifier of resource
C. Protocol
D. Server
Answer: B

---

30. SOAP uses which format?

A. JSON
B. XML
C. HTML
D. Text
Answer: B

Aptitude topics MCQ Question

Time & Work

---

1. A can complete a work in 10 days, B in 15 days. Together they take?

A. 5 days
B. 6 days
C. 7 days
D. 8 days

Answer: B
Solution:
A = 1/10, B = 1/15
Total = 1/10 + 1/15 = (3+2)/30 = 5/30 = 1/6
Time = **6 days**

---

2. A does a job in 12 days, B in 18 days. How long together?

A. 6
B. 7.2
C. 8
D. 9

Answer: B
Solution:
1/12 + 1/18 = (3+2)/36 = 5/36
Time = 36/5 = **7.2 days**

---

3. A is twice as efficient as B. If B takes 12 days, A takes?

A. 4
B. 5
C. 6
D. 8

Answer: C
Solution:
Efficiency ratio = 2:1
Time ratio = 1:2
A = 12/2 = **6 days**

---

4. A can do work in 20 days. After 5 days, B joins and completes in 10 more days. B alone takes?

A. 15
B. 20
C. 25
D. 30

Answer: B
Solution:
A does 5/20 = 1/4
Remaining = 3/4
(A+B) does 3/4 in 10 days → rate = 3/40
A = 1/20
B = (3/40 - 1/20) = 1/40 → Time = **40 days**
❗ Correction → Option mismatch → closest logical = 40 (not listed)

---

5. A, B, C complete work in 6, 8, 12 days. Together?

A. 2
B. 3
C. 4
D. 5

Answer: B
Solution:
1/6 + 1/8 + 1/12 = (4+3+2)/24 = 9/24 = 3/8
Time = 8/3 ≈ **2.67 ≈ 3 days**

---

6. If 4 men do work in 8 days, 8 men take?

A. 2
B. 4
C. 6
D. 8

Answer: B
Solution:
Work = constant
Men × Days = constant
4×8 = 32
8×x = 32 → x = **4 days**

---

7. A finishes in 15 days. B is 50% more efficient. Time taken by B?

A. 8
B. 10
C. 12
D. 14

Answer: B
Solution:

Efficiency B = 1.5A
Time = 15 / 1.5 = **10 days**

---

8. A can do in 9 days, B in 18 days. Working alternately starting A → total days?

A. 10
B. 11
C. 12
D. 13

Answer: C
Solution:
A+B in 2 days = 1/9 + 1/18 = 1/6
6 cycles = 12 days

---

9. Pipe fills tank in 6 hrs, leak empties in 12 hrs. Net time?

A. 10
B. 12
C. 8
D. 6

Answer: B
Solution:
1/6 - 1/12 = 1/12
Time = **12 hrs**

---

10. 5 workers finish work in 20 days. 10 workers take?

A. 5
B. 10
C. 15
D. 20

Answer: B
Solution:
5×20 = 100
10×x = 100 → x = **10 days**

Mixture & Alligation

---

11. Mix 10L water in 40L milk. Ratio?

A. 4:1
B. 1:4
C. 2:5
D. 5:2

Answer: A

Solution:
Milk = 40L, Water = 10L
Ratio (Milk : Water) = 40 : 10
Divide by 10 → **4 : 1**

---

12. 30L solution has 20% alcohol. Alcohol?

A. 5
B. 6
C. 7
D. 8

Answer: B

Solution:
Alcohol = 20% of 30
= (20/100) × 30
= **6L**

---

13. 10L milk, 20% replaced with water. Pure milk left?

A. 6
B. 7
C. 8
D. 9

Answer: C

Solution:
20% of 10L = 2L removed (pure milk)
Remaining milk = 10 − 2 = **8L**

---

14. Ratio 3:2, total 25L. Milk?

A. 10
B. 12
C. 15
D. 18

Answer: C

Solution:
Total parts = 3 + 2 = 5
Milk = (3/5) × 25
= **15L**

---

15. Mix 10L (₹20/L) and 10L (₹30/L). Avg price?

A. 22
B. 24
C. 25
D. 28

Answer: C

Solution:
Total cost = (10×20) + (10×30) = 200 + 300 = 500
Total quantity = 20L
Average price = 500 / 20 = **₹25/L**

---

16. Alligation: 10% and 30% → 20%. Ratio?

A. 1:1
B. 2:1
C. 1:2
D. 3:2

Answer: A

Solution (Alligation Method):
Higher − Mean = 30 − 20 = 10
Mean − Lower = 20 − 10 = 10
Ratio = 10 : 10 = **1 : 1**

---

17. Add water to reduce 50% to 25% in 10L. Water added?

A. 5
B. 10
C. 15
D. 20

Answer: B

Solution:
Initial milk = 50% of 10 = 5L
Let final volume = x

25% of x = 5
x = 20L

Water added = 20 − 10 = **10L**

---

18. Ratio 2:3, total 50L. Water?

A. 20
B. 25
C. 30
D. 35

Answer: C

Solution:
Total parts = 2 + 3 = 5
Water = (3/5) × 50
= **30L**

---

19. Replace 10% twice in 100L. Milk left?

A. 81
B. 80
C. 82
D. 79

Answer: A

Solution:
Formula: Remaining = Initial × $(1 - x)^n$

Here, x = 10% = 0.1, n = 2

Milk left = $100 \times (0.9)^2$
$= 100 \times 0.81$
= **81L**

---

20. Mix 20% and 40% → 30%. Ratio?

A. 1:1
B. 2:1
C. 1:2
D. 3:2

Answer: A

Solution (Alligation):
40 − 30 = 10
30 − 20 = 10

Ratio = 10 : 10 = **1 : 1**

Permutations & Combinations

---

21. 5! = ?

A. 60
B. 100
C. 120
D. 150

Answer: C

Solution:
Factorial means multiplication of all positive integers up to that number:
$5! = 5 \times 4 \times 3 \times 2 \times 1 =$ **120**

---

22. Number of ways to arrange 3 letters?

A. 3
B. 6

C. 9
D. 12

Answer: B

Solution:
Arrangement of 3 distinct letters = 3!
= 3 × 2 × 1 = **6 ways**

---

23. 5P2 = ?

A. 10
B. 20
C. 25
D. 30

Answer: B

Solution:
Formula: nPr = n! / (n−r)!
5P2 = 5! / 3!
= (5 × 4 × 3!) / 3!
= 5 × 4 = **20**

---

24. 5C2 = ?

A. 10
B. 20
C. 5
D. 15

Answer: A

Solution:
Formula: nCr = n! / [r!(n−r)!]
5C2 = 5! / (2! × 3!)
= (5 × 4 × 3!) / (2 × 1 × 3!)
= 20 / 2 = **10**

---

25. Arrange "ABC"?

A. 3
B. 6
C. 9
D. 12

Answer: B

Solution:
All letters are distinct
Total arrangements = 3!
= 3 × 2 × 1 = **6**

---

26. 6C3 = ?

A. 15
B. 20
C. 30
D. 60

Answer: B

Solution:
6C3 = 6! / (3! × 3!)
= (6 × 5 × 4 × 3!) / (3! × 3!)
= (6 × 5 × 4) / (3 × 2 × 1)
= 120 / 6 = **20**

---

27. Ways to choose 2 from 4?

A. 4
B. 5
C. 6
D. 8

Answer: C

Solution:
4C2 = 4! / (2! × 2!)
= (4 × 3 × 2!) / (2 × 1 × 2!)
= 12 / 2 = **6**

---

28. Arrange 4 objects → ?

A. 12
B. 16
C. 24
D. 32

Answer: C

Solution:
Arrangement of 4 distinct objects = 4!
= 4 × 3 × 2 × 1 = **24**

---

29. 4P2 = ?

A. 8
B. 10
C. 12
D. 16

Answer: C

Solution:
4P2 = 4! / (4−2)!
= 4! / 2!
= (4 × 3 × 2!) / 2!
= 4 × 3 = **12**

---

30. Ways to select 3 from 5?

A. 10
B. 15
C. 20
D. 25

Answer: A

Solution:
5C3 = 5! / (3! × 2!)
= (5 × 4 × 3!) / (3! × 2 × 1)
= 20 / 2 = **10**

Programming practice

Example of validation using JavaScript

---

1. Name Validation

Description:
Ensures the name contains only letters and is of valid length.

Example:

- Only letters (uppercase/lowercase) allowed.
- Minimum 2 characters, maximum 30 characters.

Explanation:
This prevents numbers or special characters in the name field.

Example:
"John" → valid
"John123" → invalid

---

2. Email Validation

Description:
Checks if the email is in a proper format like example@domain.com.

Example:

- Must contain @ and . after @.

Explanation:
Helps avoid invalid email submissions.

Example:
"user@example.com" → valid
"userexample.com" → invalid

---

3. Phone Number Validation

Description:
Ensures phone number is numeric and has a valid length (e.g., 10 digits).

Example:

- Only numbers allowed.
- Length must be exactly 10 digits.

Example:
"9876543210" → valid
"98765abc10" → invalid

---

4. Password Validation

Description:
Ensures passwords are strong and meet minimum security criteria.

Example:

- Minimum 8 characters
- At least one uppercase, one lowercase, one digit, and one special character

Example:
"Pass@123" → valid
"password" → invalid

---

5. Confirm Password Validation

Description:
Checks if the confirm password field matches the original password.

Example:

- Ensures user did not mistype password.

Example:
password = "Pass@123"
confirmPassword = "Pass@123" → valid
confirmPassword = "Pass123" → invalid

---

6. Age Validation

Description:
Validates if age is numeric and falls within a specific range.

Example:

- Acceptable age: 18–60

Explanation:
Prevents unrealistic or invalid age entries.

Example:
25 → valid
15 → invalid

---

7. URL Validation

Description:
Checks if the input is a valid URL.

Example:

- Should start with http:// or https://
- Should have a domain name

Example:
"https://www.example.com" → valid
"www.example" → invalid

---

8. ZIP/Postal Code Validation

Description:
Validates ZIP or postal code format.

Example:

- Only digits allowed
- Must be 5 or 6 digits depending on country

Example:
"560001" → valid
"56A001" → invalid

---

9. Credit Card Number Validation

Description:
Validates a credit card number using length and pattern checks.

Example:

- Only numbers
- 16 digits

Example:
"1234567812345678" → valid
"1234-5678-1234-5678" → invalid

---

10. Checkbox/Terms Acceptance Validation

Description:
Ensures user accepts terms and conditions.

Example:

- Must check the box before submission

Example:
termsChecked = true → valid
termsChecked = false → invalid

11. Parse JSON

Description:
Parsing JSON means converting a JSON-formatted string into a JavaScript object so that it can be used in a program.

Explanation:
JSON data usually comes from APIs or storage as a string. To access properties like name or age, it must be converted into an object.

Example:
If you receive data like "{"name":"John"}", after parsing, you can access the name as a normal object property.

---

12. Stringify Object

Description:
Stringifying means converting a JavaScript object into a JSON string.

Explanation:
This is useful when sending data to a server or storing it in browser storage, since only strings can be stored or transmitted easily.

Example:
An object containing name and age can be converted into a JSON string format like {"name":"John","age":25}.

---

13. Access JSON Property

Description:
Accessing a JSON property means retrieving a value from a JSON object using its key.

Explanation:
Once JSON is parsed into an object, its properties can be accessed just like any normal object.

Example:
If an object has a key called "name", you can retrieve the value associated with it, such as "John".

---

14. Modify JSON

Description:
Modifying JSON means adding, updating, or removing properties from a JSON object.

Explanation:
After parsing, the JSON becomes a normal object, so new properties can be added or existing ones can be changed.

Example:
You can add a new property like age with a value such as 25 to an existing object.

---

15. Nested JSON

Description:
Nested JSON refers to JSON objects inside other objects.

Explanation:
To access deeply nested values, you must go step by step through each level.

Example:
If a user object contains an address object, and that address has a city, you access the city by going through user → address → city.

JSON Array + Objects (Q16–Q30)

---

16. Find Employee by ID

Description:
Locate a specific employee object from an array using a unique identifier like id.

Explanation:
Each employee has a unique id, so you search the array and return the matching object.

Example:
From a list of employees, find the one whose id is 5 and retrieve their details.

---

17. Filter Employees by Salary

Description:
Get a list of employees whose salary meets a certain condition.

Explanation:
Filtering means selecting only those elements that satisfy a condition.

Example:
From the employee list, return only those earning more than 50,000.

---

18. Get All Names

Description:
Extract specific values (like names) from each object in an array.

Explanation:
Instead of full objects, you create a new list containing only names.

Example:
From employee objects, create a new list such as ["Amit", "Neha", "Raj"].

---

19. Count Employees

Description:
Find the total number of employee objects in the array.

Explanation:
Arrays have a built-in way to determine how many elements they contain.

Example:
If there are 10 employee objects, the count is 10.

---

20. Find Highest Salary

Description:
Identify the employee with the maximum salary.

Explanation:
You compare salaries of all employees and find the highest one.

Example:
From the list, find the employee earning 80,000 if that is the highest.

---

21. Find Lowest Salary

Description:
Identify the employee with the minimum salary.

Explanation:
Similar to maximum, but here you find the smallest value.

Example:
Find the employee earning 30,000 if it is the lowest.

---

22. Sort Employees by Salary

Description:
Arrange employees in order based on salary.

Explanation:
Sorting can be ascending (low to high) or descending (high to low).

Example:
Employees can be ordered starting from 30,000 up to 80,000.

---

23. Group by Department

Description:
Organize employees based on their department.

Explanation:
Employees with the same department are grouped together.

Example:
All IT employees go into one group, HR into another, Finance into another.

---

24. Check Employee Exists

Description:
Verify whether an employee with a specific id exists in the array.

Explanation:
Instead of returning the object, this returns true or false.

Example:
Check if there is any employee with id 3.

---

25. Remove Employee

Description:
Delete an employee object from the array based on a condition.

Explanation:
You create a new list excluding the unwanted employee.

Example:
Remove the employee whose id is 4 from the list.

---

26. Update Employee

Description:
Modify an existing employee's details.

Explanation:
You locate the employee and update a specific field like salary.

Example:
Change salary of employee with id 2 from 60,000 to 65,000.

---

27. Add New Employee

Description:
Insert a new employee object into the array.

Explanation:
New data can be appended to the existing list.

Example:
Add a new employee with id 11 and relevant details.

---

28. Find Duplicate IDs

Description:
Detect if multiple employees share the same id.

Explanation:
Duplicate ids indicate data inconsistency and should be identified.

Example:
If two employees have id 5, that is a duplicate.

---

29. Convert Array to JSON

Description:
Transform a JavaScript array into a JSON string.

Explanation:
Useful for sending data to APIs or storing it.

Example:
Convert employee array into a string format like [{"id":1,"name":"Amit"}].

---

30. Loop Through JSON Array

Description:
Iterate over all elements in a JSON array.

Explanation:
Looping allows you to perform operations on each item.

Example:
Print the name of every employee one by one.